



优方软件包架构说明

版本：V1.00 日期：2022-11-10

www.bestsolution.com



目录

1 软件包架构说明	3
1.1 主程序 MAIN_PROGRAM 介绍.....	3
1.2 外部函数介绍	3
1.3 外部函数工作说明	3
2 软件包流程图	4
2.1 工作流程图	4
2.2 工作流程图说明	5
2.3 休眠流程图	6
2.4 休眠流程图说明	7
3 软件包客制程序范例	8
3.1 创建 C/ASM 程序档案.....	8
3.2 编写 EXT 档案	9
3.3 配置 INC 档案	10
4 软件包结构图	11
4.1 优方软件包文件夹结构	11
4.2 V512 软件包引用关系图	13
4.3 新版软件包优点	14



1 软件包架构说明

1.1 主程序 MAIN_PROGRAM 介绍

MAIN_PROGRAM 为软件包的核心控制程序，可以实现 IC 硬件初始化，执行各功能的外部函数以及对 IC 工作与休眠的流程控制。用户可将自定义的功能函数 (如触控功能、A/D 采集功能、串口通信等等功能) 加入到 MAIN_PROGRAM，主程序会通过软件流程控制依次实现各个功能。

1.2 外部函数介绍

V512 软件包共有 5 个外部功能函数，每个外部功能函数执行的功能都不相同。用户可根据实际需要自定义函数加入以下五个外部功能函数之中：

① EXTEND_FUNCTION_INITIAL()

执行外部初始化函数，只运行一次。

② EXTEND_FUNCTION()

执行外部工作函数，若没开启省电模式，程序会在此循环执行。

③ EXTEND_FUNCTION_HALT_PREPARE()

准备进入待机前执行的函数。

④ EXTEND_FUNCTION_HALT_WAKEUP()

待机模式被中断、I/O 唤醒之后执行的函数。

⑤ EXTEND_FUNCTION_RETURN_MAIN()

待机模式切回工作模式时执行的函数。

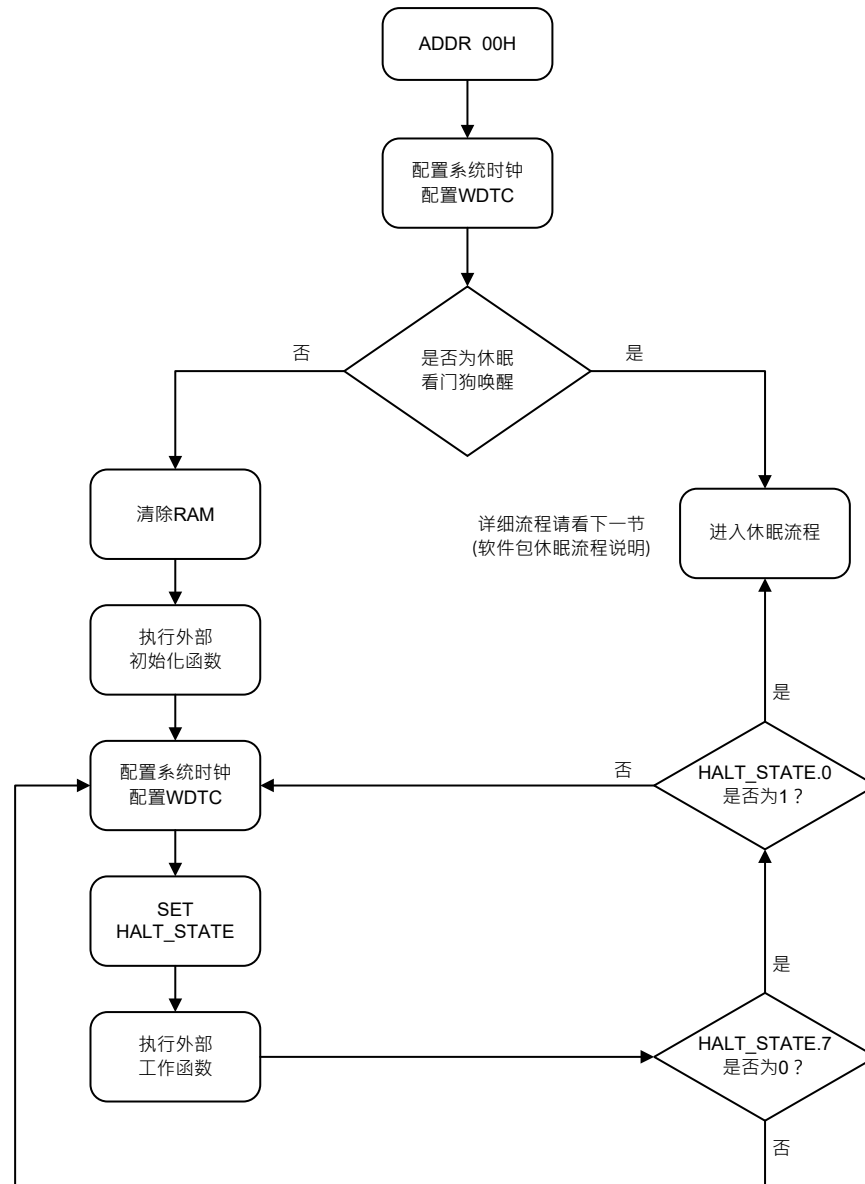
1.3 外部函数工作说明

当 MAIN_PROGRAM 在执行某外部函数时，软件包会从 EXTEND_FUNCTION_1A_XXXX 到 EXTEND_FUNCTION_2F_XXXX 依次向下执行，其中的 1A~2F 表示该外部函数区不同的入口。用户可在 MAIN_PROGRAM.INC 中为自定义的功能函数设置不同的函数入口，当软件包执行该外部功能函数时，会根据 1A~2F 的顺序依次执行。(V512 软件包的触控包也是属于自定义功能函数，可以不添加触控功能)



2 软件包流程图

2.1 工作流程图





2.2 工作流程图说明

2.2.1 初始化流程说明

IC 上电或者复位的时候软件包会从 00H 往下执行，在配置系统时钟和看门狗溢出时间后，程序会根据 PDF 标志位与 TO 标志位来判断，当这两个标志位均为 1 时表示 IC 在休眠被看门狗溢出唤醒。

如果软件包是休眠情况下被看门狗唤醒，程序会直接走右边的路线进入休眠流程，上电或者其他情况导致的复位则会走左边的初始化流程，清 RAM 并执行所有的初始化函数，然后进入工作流程。

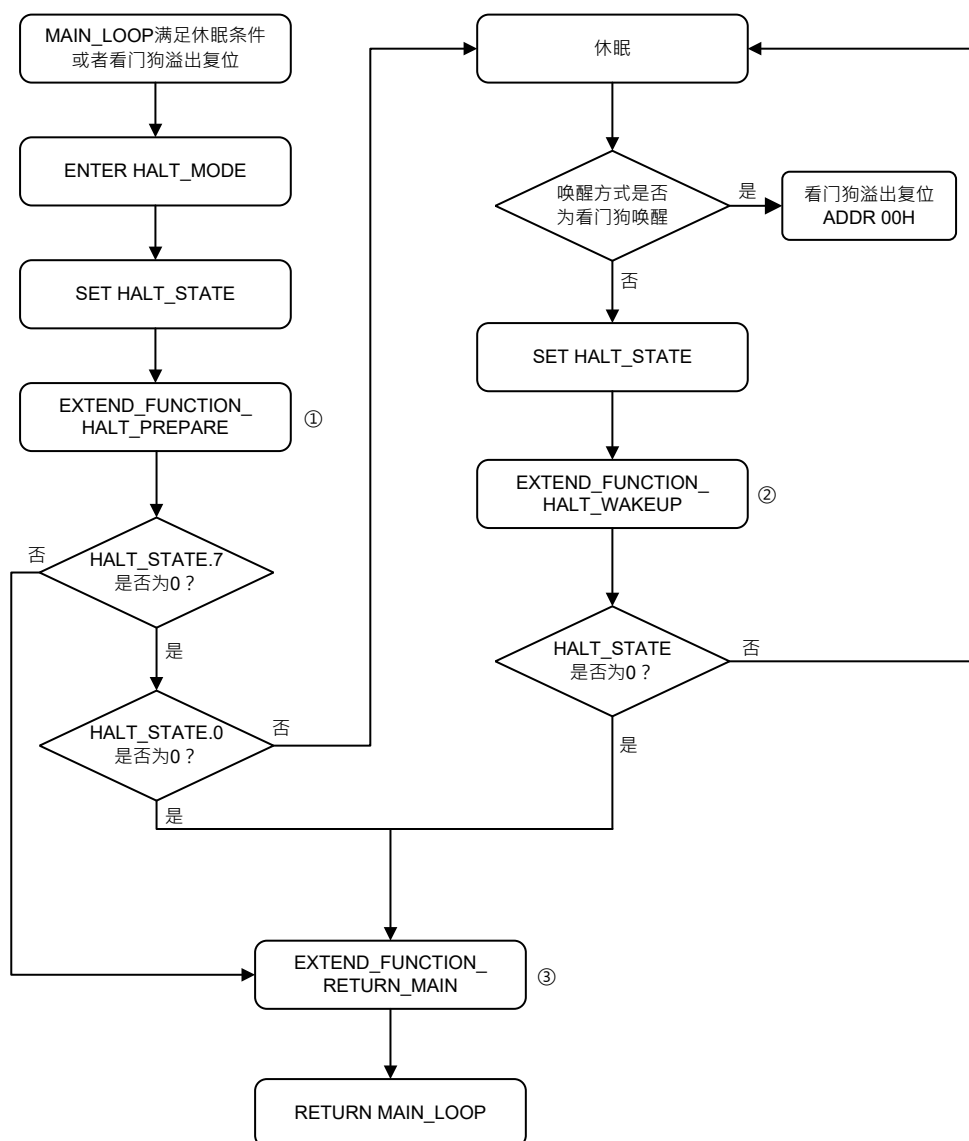
2.2.2 工作流程说明

当软件包初始化完成后，软件包会进入工作流程 MAIN_LOOP 中。若软件包未开启省电模式，软件包会循环执行 MAIN_LOOP 中的工作流程。在配置系统时钟和看门狗溢出时间之后将休眠标志位 HALT_STATE 置一，执行用户所有的外部工作函数：EXTEND_FUNCTION。

执行完外部工作函数后，软件包会根据休眠标志位 HALT_STATE 判断程序是否满足休眠条件。若开启休眠模式的话触控包底层会对 HALT_STATE 操作，让软件包满足休眠条件后可以进入休眠。若未开启休眠模式用户需要自己在外部工作函数对 HALT_STATE 操作才能进入休眠，若用户无操作软件包将会循环执行工作流程。



2.3 休眠流程图





2.4 休眠流程图说明

2.4.1 休眠标志位

V512 软件包若开启省电模式，程序会根据休眠标志位 `HALT_STATE` 控制软件包的休眠流程。以下为休眠标志位最低位与最高位的解释说明：

1. `HALT_STATE.0` (可读写)

该位为程序能否休眠判断位。若该位为 1 表示程序满足休眠条件，之后会进入休眠。若该标志位为 0 则表示程序不满足休眠条件，之后程序会返回工作流程。

2. `HALT_STATE.7` (可读写)

该位为休眠标志位 RAM 操作有效位。若该标志位为 0 则视为休眠标志位有效，软件包会根据 `HALT_STATE.0` 决定软件包能否进入休眠。若该标志位为 1 则操作无效，软件包将不会根据休眠判断位进行流程控制。因此在修改休眠标志位时必须将 `HALT_STATE.7` 清零。

2.4.2 休眠函数说明

软件包进入休眠流程后，会根据流程图执行三个外部休眠函数。每个外部休眠函数的执行方式如下所示：

① `EXTEND_FUNCTION_HALT_PREPARE()`

外部休眠准备函数：IC 进入休眠流程后执行

② `EXTEND_FUNCTION_HALT_WAKEUP()`

外部休眠唤醒函数：IC 从休眠模式被中断、I/O 唤醒之后执行

③ `EXTEND_FUNCTION_RETURN_MAIN()`

外部休眠返回工作函数：IC 从休眠模式返回工作模式时执行

2.4.3 休眠流程说明

在执行外部休眠准备与唤醒函数前，软件包会先执行 `SET HALT_STATE` 的操作。即，将 `HALT_STATE.7` 与 `HALT_STATE.0` 置一，用户可在自定义的外部休眠函数中对 `HALT_STATE` 标志位进行操作。软件包会根据用户执行外部函数后的 `HALT_STATE` 来对程序进行休眠流程控制。

若用户在外部的休眠准备函数中没有对 `HALT_STATE` 操作，软件包将无法休眠。而是直接返回外部工作函数 `MAIN_LOOP`，若用户在外部的休眠唤醒函数中没有对 `HALT_STATE` 操作，IC 在执行完所有外部休眠唤醒函数后将继续进入休眠。



3 软件包客制程序范例

以在优方的软件包中添加一个 A/D 采集客制程序为例，用户可根据以下的步骤创建 C/ASM、EXT、INC 档案并添加到软件包架构当中，从而实现制作一个带 A/D 采集的软件包。

3.1 创建 C/ASM 程序档案

新建一个 C/ASM 文档并在其中编写 A/D 采集程序，并将初始化函数与工作函数分开编写，软件包上电执行完所有的初始化函数，会循环执行所有的工作函数，C 语言的范例代码如下：

```
#include "AD_FUNCTION.INC"
//=====
//*****
//=====

unsigned char AD_VALUE_L = 0;
unsigned char AD_VALUE_H = 0;

void AD_FUNCTION_INITIAL()
{
// A/D//
_sadc0 = 0b00010000; //設置輸入通道為AN0
_sadc1 = 0b00001010; //外部通道,UDD,Fsys/4
_ace0 = 1; //设置PC0为AN0通道
_adcen = 1;
}

//=====
//*****
//=====

void AD_FUNCTION_WORK()
{
_start = 0;
_start = 1;
_start = 0;
while(_adbz);
AD_VALUE_L = _sadol;
AD_VALUE_H = _sadoh;
}
```




3.2 编写 EXT 档案

为了让软件包中功能文档的函数与参数能够被其他的档案使用，可通过为每一个 C/ASM 档案添加对应的 EXT 档案。如 MAIN_PROGRAM.ASM、USER_PROGRAM.ASM (C)、BSXXXXXC_CTOUCH.OBJ 这些文档都有一一对应的 EXT 档案，这些 EXT 档案中会存放自身函数或变量的外部声明，只要在头文件中包含这些 EXT 档案就可以直接调用这些功能文档内的函数与变量。

我们在添加客制功能的时候也需要添加一个 EXT 文档，让其他文档能够通过这个 EXT 文档来调用我们添加的客制功能函数与参数，下面为 EXT 文档声明外部函数与变量的方式：

声明汇编函数：EXTERN _XXX :near

声明汇编参数：EXTERN _A :BYTE

声明 C 语言函数：extern void XXX();

声明 C 语言参数：extern unsigned char A;

为了让 EXT 档案能同时识别汇编与 C 语言的声明，可以通过使用 `;;;/*` 与 `;;;*/` 来划分 C 语言与汇编程序区域，当编译程序为汇编时，`/*` 与 `*/` 将会被注释，从而能够编译注释符之间的程序。反之将跳过 `/*` 与 `*/` 注释符之间的汇编程序（一个程序 / 变量可同时进行汇编和 C 语言的外部声明）。

将 A/D 采集程序的初始化、工作函数以及储存 A/D 值的两个全局变量 AD_VALUE 在 EXT 档案中进行声明，范例代码如下图红框所示：（可以从别的 EXT 文档复制并修改 XXX_DEF、函数名、参数名）

```

;;;/*
ifndef AD_FUNCTION_DEF
    EXTERN  _AD_FUNCTION_INITIAL    :NEAR
    EXTERN  _AD_FUNCTION_WORK      :NEAR

    EXTERN  _AD_VALUE_H            :BYTE
    EXTERN  _AD_VALUE_L            :BYTE
endif
;;;*/

;;;/*
;;;/* C exported function,register */
;;;/*
;;;/*
#ifdef _INCLUDE_C_
;;;/*
;;;/*
#endif AD_FUNCTION_DEF
    extern void  AD_FUNCTION_INITIAL();
    extern void  AD_FUNCTION_WORK();

    extern unsigned char  AD_VALUE_H;
    extern unsigned char  AD_VALUE_L;
#endif
;;;/*
;;;/*
endif
;;;*/

```



3.3 配置 INC 档案

优方软件包内所有的 EXT 档案都会包含在 GLOBE_INCLUDE.INC 中，同时将对应的 MCU 硬件资源存在 GLOBE_VARIES.INC 中。因此软件包内所有的 INC 文档只要包含这两个头文件便可以包含所有的 EXT 文档以及对应 MCU 硬件资源。

但是这样会让程序也包含自身的 EXT 档案从而对调用自身函数的外部声明出现报错。因此需要在 EXT 档案中使用预编译 ifndef/endif 语句来规避这个问题。首先在对应的 INC 文档中定义一个 XXX_DEF 作为标记，之后在对应的 EXT 档案中使用预编译判断是否是自身的头文件，如果是自身的头文件就不进行外部声明，从而规避掉这个错误。以下为配置步骤：

步骤一：在 AD_FUNCTION.INC 文档中定义 AD_FUNCTION_DEF，并包含软件包的两个全局头文件 GLOBE_VARIES.INC 和 GLOBE_INCLUDE.INC，实现全局变量的调用。

```
#define AD_FUNCTION_DEF
#include "..\GLOBE\GLOBE_VARIES.INC"
#include "..\GLOBE\GLOBE_INCLUDE.INC"
```

步骤二：在 AD_FUNCTION.EXT 文档中使用 ifndef/endif 语句，避免由于对自身函数与变量的外部声明造成编译错误。

```
;;;;/*
ifndef AD_FUNCTION_DEF
    EXTERN  _AD_FUNCTION_INITIAL      :NEAR
    EXTERN  _AD_FUNCTION_WORK        :NEAR

    EXTERN  _AD_VALUE_H              :BYTE
    EXTERN  _AD_VALUE_L              :BYTE
endif
;;;;*/

;;;;/* ----- */
;;;;/* C exported function,register */
;;;;/* ----- */
;;;;/*
ifdef _INCLUDE_C_
;;;;*/
;;;;//-----
#endif
    extern void  AD_FUNCTION_INITIAL();
    extern void  AD_FUNCTION_WORK();

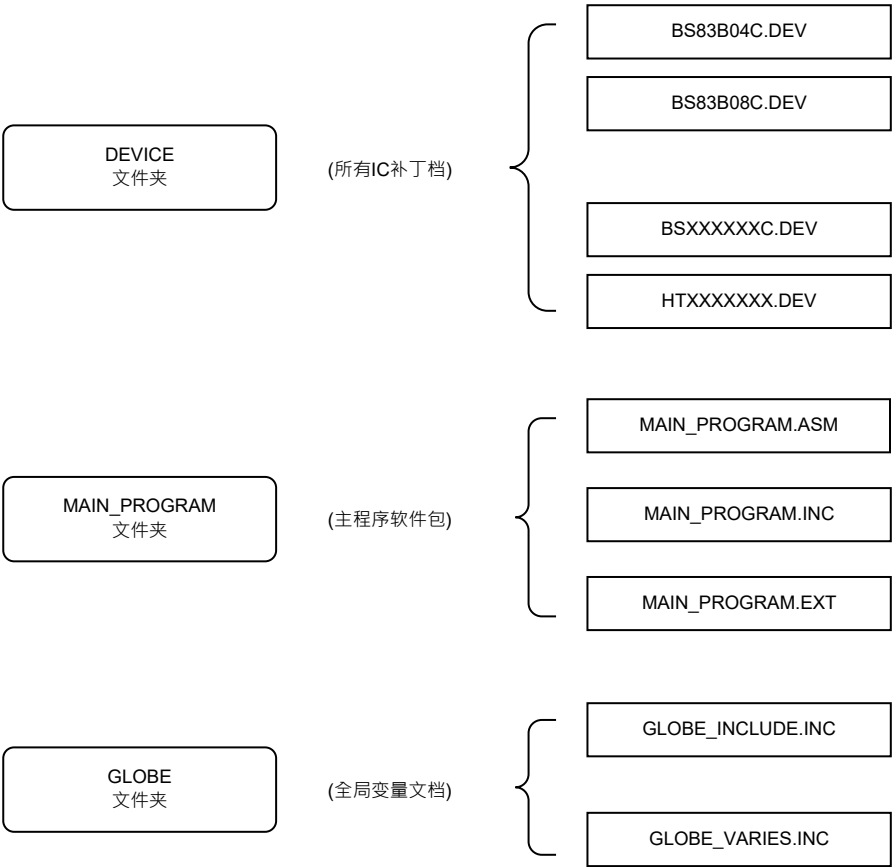
    extern unsigned char  AD_VALUE_H;
    extern unsigned char  AD_VALUE_L;
#endif
;;;;//-----
;;;;/*
endif
;;;;*/
```



4 软件包结构图

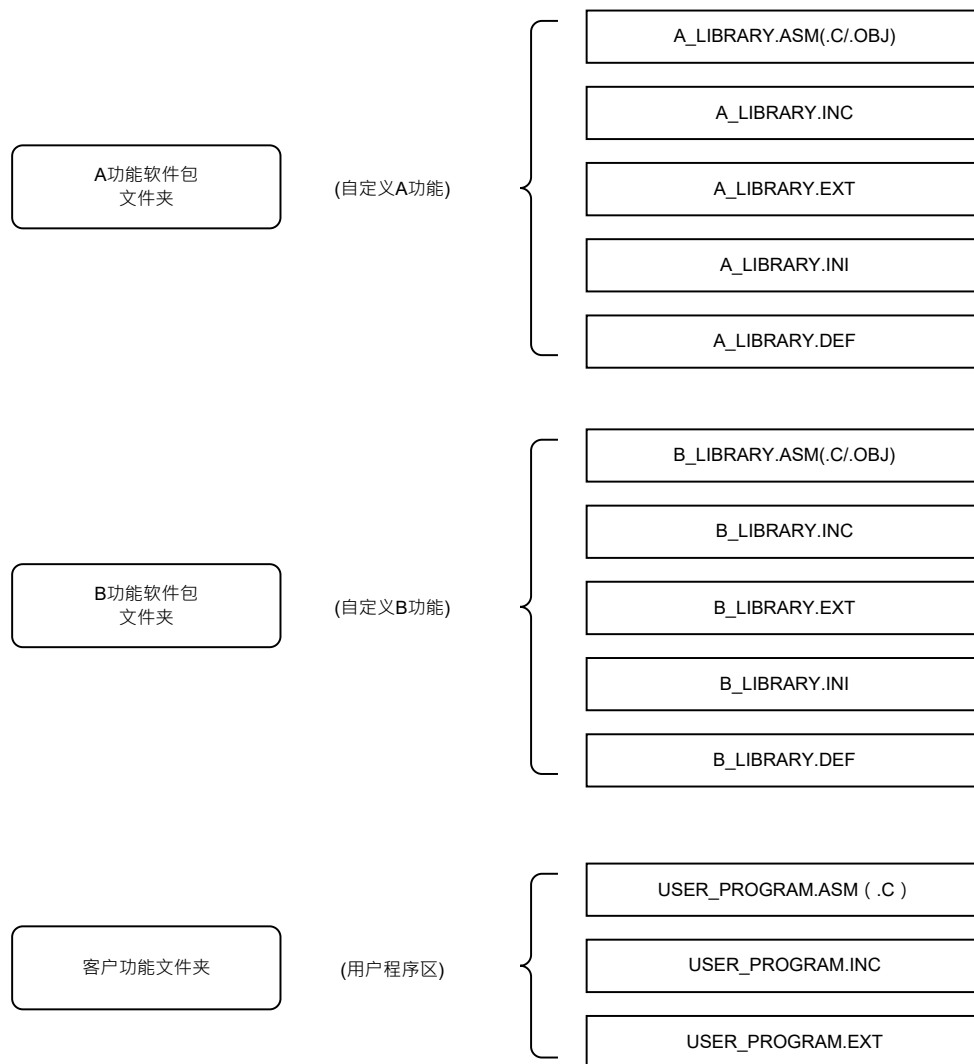
4.1 优方软件包文件夹结构

V512 版本软件包的文件夹分为两个部分，第一部分是软件包架构相关文件夹，用于存放软件包正常工作的必要文件，第二部分是用户自定义程序文件夹，用于编写软件包的客制功能。如下图所示：



软件包架构相关文件夹

1. **DEVICE 文件夹：**用于存放对应 IC 的 DEV 文档，其中的 DEV 文档会对 MCU 内部资源进行定义宣告的，因此软件包在使用不同 IC 时只需要修改该文件夹的 DEV 档案即可。
2. **MAIN_PROGRAM 文件夹：**用于存放软件包主程序，通过执行 MAIN_PROGRAM.ASM 中的程序实现软件包的运作，用户可通过 MAIN_PROGRAM.INC 将自定义的外部函数加入到主程序的循环中，EXT 档案的则是对主程序的标志位进行外部声明。
3. **GLOBE 文件夹：**用于存放软件包全局变量文档。第一个是 GLOBE_INCLUDE.INC，其包含了软件包所有的 EXT 文档，等于包含了软件包所有的共用函数与变量的外部声明。第二是 GLOBE_VARIES.INC 其包含 MCU 的硬件宣告 DEV 文档，等于包含了软件包对应 MCU 的硬件信息。



用户自定义程序文件夹

1. 功能软件包文件夹：用于存放用户编写的功能函数，例如触控包就属于一种功能软件包。由于触控包文档为 .OBJ 形式，因此一般的方法无法修改其中的变量，需通过 INI 文档来间接修改其中的参数 (MAIN_PROGRAM.INC 中也需要包含对应的 INI 文档)。根据上文 3.2 和 3.3 章添加客制功能范例配置 INC 文档和 EXT 文档后便可将客制功能加入软件包中。

DEF 文档为程序扩充文档，例如编写 IIC 程序获取 0X10 的数据，但是想要获取 0X12 的地址时需要重新配置 IIC 的程序，如果将获取 0X10 和获取 0X12 的程序封装为 DEF 文档后，可根据实际情况外挂不同的 DEF 文档。该文档可根据需求选择是否添加。

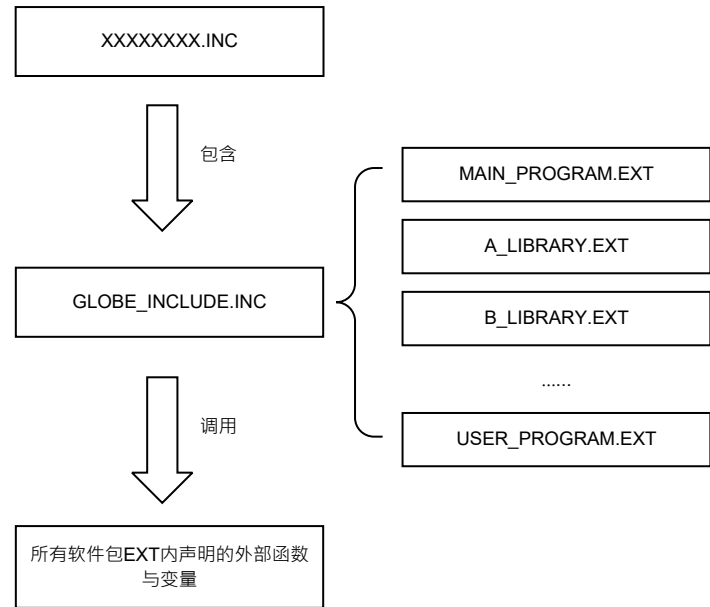
2. 客户功能文件夹：用于存放客户的程序代码，用户可在 USER_PROGRAM.ASM(.C) 中编写自己的程序，同时可以直接调用其他文档的函数与变量为自身使用。



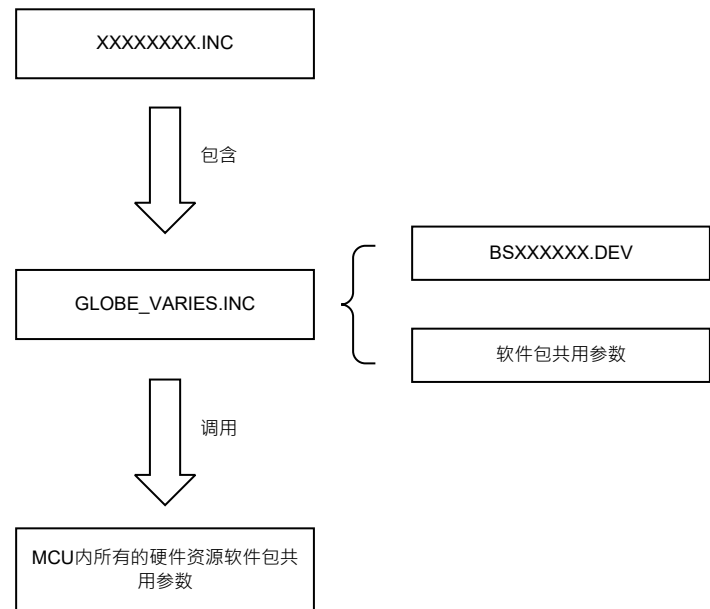
4.2 V512 软件包引用关系图

V512 软件包以 GLOBE 文件夹中的 GLOBE_INCLUDE.INC、GLOBE_VARIES.INC 为纽带，通过包含这两个头文件实现对软件包各个文档的引用与连接。

1. GLOBE_INCLUDE.INC 内包含了所有的 EXT 文档，软件包内的文档只要包含了该头文件后就可以间接调用软件包内所有声明过的外部函数与变量，具体的关系图如下

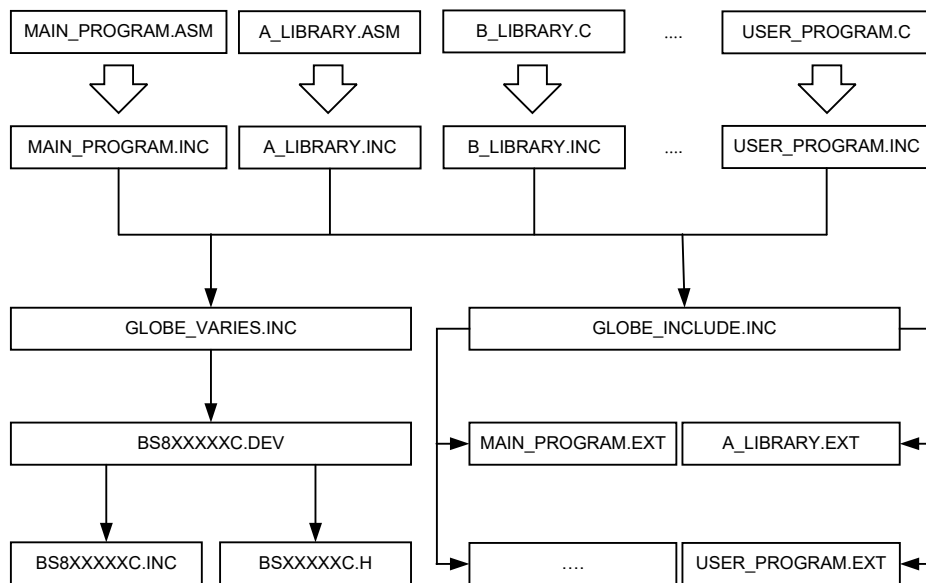


2. GLOBE_VARIES.INC 内包含 MCU 的资源 DEV 档，软件包内的头文件只要包含了该头文件后就可以包含软件包内的对应 MCU 的硬件资源与共用参数，具体的关系图如下：





3. 使用 V512 软件包新增功能代码时，只需根据 3.2 和 3.3 章教程配置 EXT 文档与 INC 文档并加入 MAIN_PROGRAM 之后，在自身 INC 文档包含 GLOBE 文件夹内的这两个纽带文件即可完成功能移植，无需改动原有配置的头文件。关系图如下所示：



软件包无论新增汇编或 C 程序，只需要将配置的 EXT 的头文件放入 GLOBE_INCLUDE.INC 中，原有的程序便可以直接调用新加入的程序的功能或参数，从而让移植新功能或程序更加便捷。

4.3 新版软件包优点

优方软件包具有完整的程序架构，还具备极强的可移植性。当用户需要添加 C 语言或汇编的客制功能时，无需对原有的框架进行大幅整改，只需要配置 EXT 文档以及修改关键的头文件就能将新功能添加进软件包中。

该版软件包摆脱了触控包束缚，用户可以不使用触控包功能函数，让该软件包不仅仅用在触控领域，也可以运用在其他低功耗软件包领域。另外新版的软件包新增三个外部休眠函数，用户可根据实际情况来编写程序，极大的增强代码的可操作空间。以下为该版软件包新增的档案说明：

1. 新增档案说明 (.DEV)

由于不同 IC 内部寄存器的名字可能不相同，旧版本的软件包采用将所有寄存器统一命名并存在 MAIN_PROGRAM.MCU 文档内，但如果有新增 IC 的情况则需要将之前所有软件包内的 MAIN_PROGRAM.MCU 更新。

新软件包内的 DEV 文档对应自身的 MCU，如 BS83B04C、BS83B08C、BS83B12C 软件包都有独立的 DEV 文档与之一一对应，该文档会将 MCU 的寄存器统一命名供软件包使用。如果有新增 IC 只需新增一个 DEV 文档，不需要对原来的软件包进行修改。



2. 新增档案说明 (.DEF)

该文档用于扩充软件包的程序，尤其针对通信专用代码 (IIC/UART)。例如我们希望使用 IIC 获取软件包不同位置的地址，只需要编写获取特定地址的 DEF 文档。在使用时将该文档挂到软件包中，便可根据实际情况选择挂任意 DEF 文档获取不同位置数据。

3. 新增档案说明 (.INI)

该文档的作用是用来协助 OBJ 档案内的数值修改，同时解决旧版本软件包必须在 MAIN_PROGRAM 中加载触控包参数的问题。让软件包摆脱一定要使用触控包的情况。该文档的好处在于：可将软件包大多数功能移至 INI，让客户能够在 INI 文档中将不使用的功能关闭从而节省 ROM 空间，并让客户能够修改部分底层的触控参数。



Copyright© 2022 by Best Solution technology INC. All Rights Reserved.

本文件出版时优方已针对所载信息为合理注意，但不保证信息准确无误。文中提到的信息仅是提供作为参考，且可能被更新取代。优方不担保任何明示、默示或法定的，包括但不限于适合商品化、令人满意的质量、规格、特性、功能与特定用途、不侵害第三方权利等保证责任。优方就文中提到的信息及该信息之应用，不承担任何法律责任。此外，优方并不推荐将优方的产品使用在会由于故障或其他原因而可能会对人身安全造成危害的地方。优方特此声明，不授权将产品使用于救生、维生或安全关键零部件。在救生 / 维生或安全应用中使用优方产品的风险完全由买方承担，如因该等使用导致优方遭受损害、索赔、诉讼或产生费用，买方同意出面进行辩护、赔偿并使优方免受损害。优方 (及其授权方，如适用) 拥有本文件所提供信息 (包括但不限于内容、数据、示例、材料、图形、商标) 的知识产权，且该信息受著作权法和其他知识产权法的保护。优方在此并未明示或暗示授予任何知识产权。优方拥有不事先通知而修改本文件所载信息的权利。如欲取得最新的信息，请与我们联系。